

AuraTrek: Trekking Management Application

Sunny Kumar Bharti | Roll Number: 24F2007908 | Diploma 4th Term

1. Project Overview

1.1 Project Description

AuraTrek is a trekking management application designed with a reactive user interface using Vue and a backend powered by Flask-API. The application streamlines the entire trekking process, simplifying booking, tracking, and management for both users and treks.

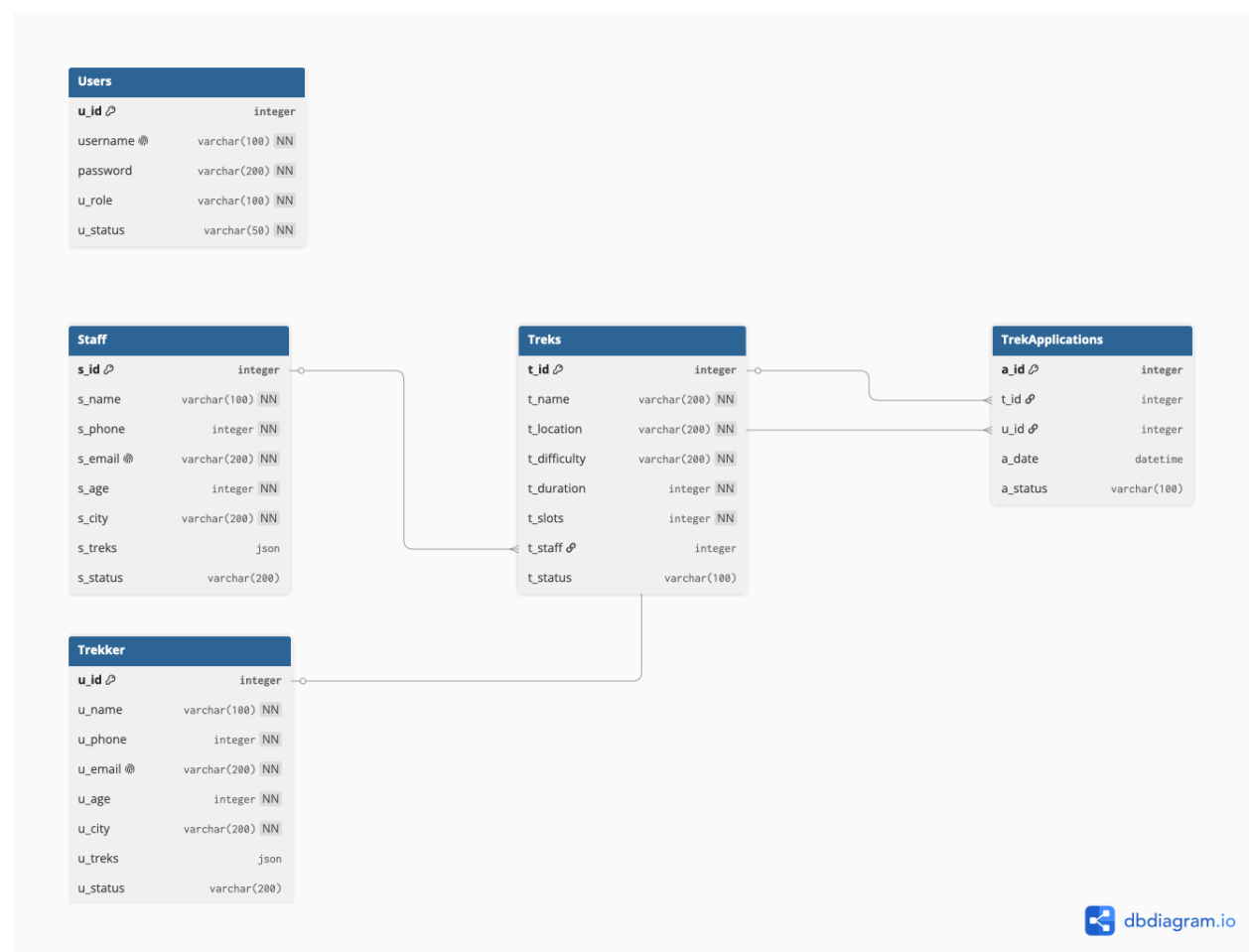
1.2 Tech Stack

Component Layer	Technology	Brief Description
Frontend Framework	Vue.js 3	Renders the modern, responsive Single Page Application (SPA) using a component-based design.
State Management	Pinia	Manages global application states and keeps user authentication persistent across views.
Build & Tooling	Vite	Provides ultra-fast asset bundling and powers the local frontend development server.
Backend Framework	Flask	Drives the core RESTful API architecture, handling request routing and business logic.
Database ORM	SQLAlchemy	Maps Python models to the relational database structure for seamless data querying.

Component Layer	Technology	Brief Description
Task Queue & Broker	Celery + Redis	Offloads time-consuming processes, like heavy CSV compilation jobs, to background workers.
Security & Auth	PyJWT	Secures API endpoints by issuing and validating role-based JSON Web Tokens (JWT).
Server-Side Caching	Flask-Caching	Optimizes response times by caching frequently requested database query results.

2. System Architecture

ER Diagram



2.1 API Architecture

The RESTful API architecture was designed to handle core operations for the AuraTrek application platform. It exposes dedicated endpoints to manage the following system workflows:

- **User Authentication & Authorization:** Formulates secure pathways for user and staff logins, provisioning dynamic access tokens.
- **Trek Roster Management:** Provides endpoints to retrieve, track, and filter lists of scheduled treks assigned to specific staff members.
- **Asynchronous Data Export Processing:** Handles triggers for user-initiated background jobs, allowing them to request their complete booking histories.

2.2 Architecture & Technical Implementation

- **Framework & Routing:** Implemented using Flask blueprints to modularize controllers into distinct routers (e.g., `user_bp`), ensuring a clean separation of concerns.
- **Database Interfacing (ORM):** Leveraged Flask-SQLAlchemy to perform relational queries and joins between data tables (such as `TrekApplications` and `Treks`) based on structural primary and foreign key mapping.
- **State & Payload Validation:** Requests consume strict JSON data payloads mapping required attributes (like email or username). To prevent unauthorized access, custom authentication decorators (`@user_required`) intercept incoming requests to validate session tokens via PyJWT.
- **Background Pipeline Delegation:** For resource-heavy tasks like compiling dataset arrays into CSV archives, the API utilizes a non-blocking architecture. It delegates the processing workload using Celery task queues with a Redis message broker, running the operation asynchronously in the background while instantly returning a 202 Accepted status code to the frontend interface.

2.3 Project Architecture & Organization

The project is structured following a clean, modular layout that separates the backend API services from the frontend user interface. The backend application is built using a Flask architecture where the core logic is divided into specialized modules. Request routing and route controllers are encapsulated inside the `routes/` directory (e.g., `routes/user.py`), keeping the API endpoints decoupled and maintainable. Database structures and data relationships are organized within `models.py`, while asynchronous, long-running operations are separated into `tasks.py`. The modern client-side application is decoupled from Flask's traditional server-side Jinja templates and is hosted entirely within the `frontend/` directory. This frontend layer utilizes a Vue.js 3 structure organized into `views/` for independent dashboard layouts, `components/` for reusable interface blocks, and a `stores/` directory powered by Pinia to handle global data states.

3. Features & Implementation

3.1 Core Platform Features

- **Role-Based Authentication:** Implemented using PyJWT, this system validates session credentials on the client side via local storage tokens and secures server endpoints using custom backend decorators (`@user_required`), cleanly separating standard users from staff members.
- **Relational Database Management:** Built with Flask-SQLAlchemy, this layer links structural models like Users, Trekker, Staff, and Treks through strict foreign keys, facilitating complex data operations like

automatic cascade deletions (cascade="all, delete-orphan") on trek registrations.

- Interactive Dashboards: Created using reactive components in Vue.js 3, providing staff members with optimized visibility into their assigned trek pipelines and users with structural management tools over their applications.

3.2 Advanced Performance Features

- Asynchronous Background Processing: Designed using a Celery execution engine paired with a Redis message broker. Heavy resource operations—such as converting dataset matrices into structured CSV sheets—are offloaded dynamically to background worker pools, keeping the primary HTTP response pipeline completely non-blocking.
- Automated Email Dispatch Systems: Integrated within the background worker task loop to immediately handle communication pipelines. Once an asynchronous job completes, the backend triggers an automated email delivery workflow, attaching generated reports securely to the user's validated mailbox.
- Server-Side Data Caching: Enhanced utilizing Flask-Caching to minimize direct database read operations for static data pools, lowering server processing overhead and decreasing UI rendering latency.

Video Link :

[Drive Link for Video](#)